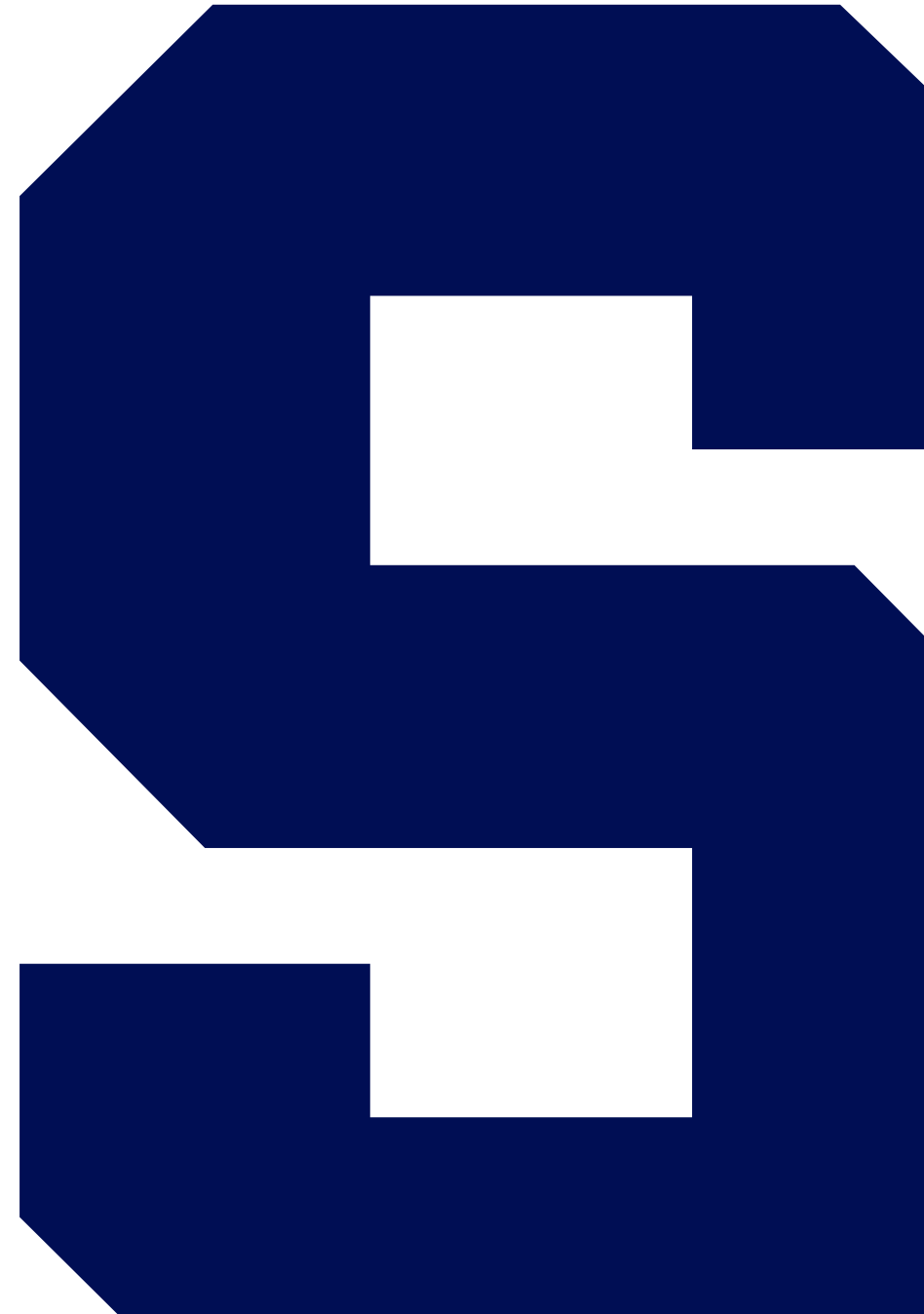
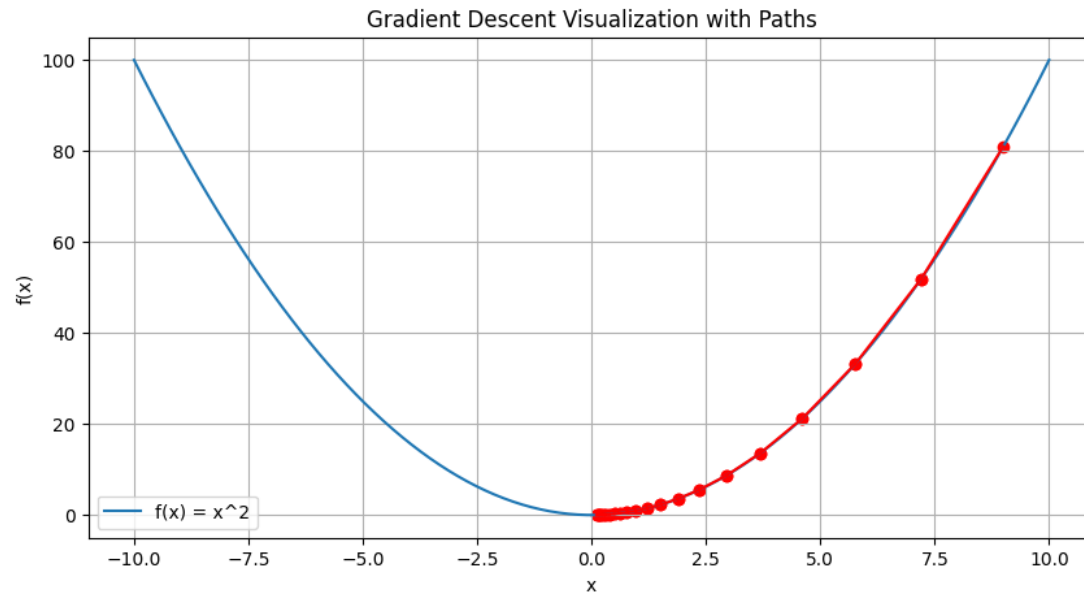


Learning Rate Schedules



The Learning Rate's Role in Backpropagation

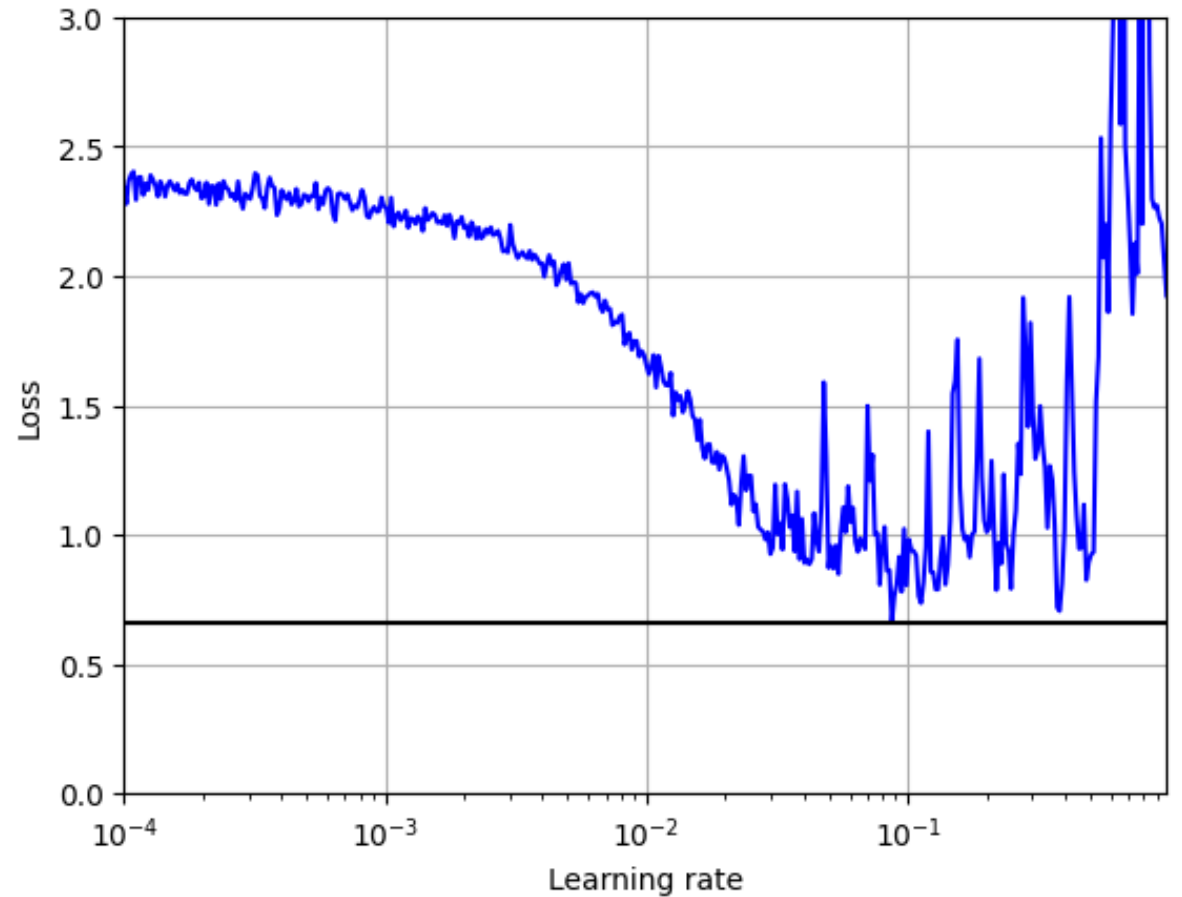
The learning rate is one of the most important hyperparameters when training a neural network.



$$w_{new} = w - \alpha * \frac{\delta L}{\delta w}$$
$$b_{new} = b - \alpha * \frac{\delta L}{\delta b}$$

Identifying a Good Learning Rate

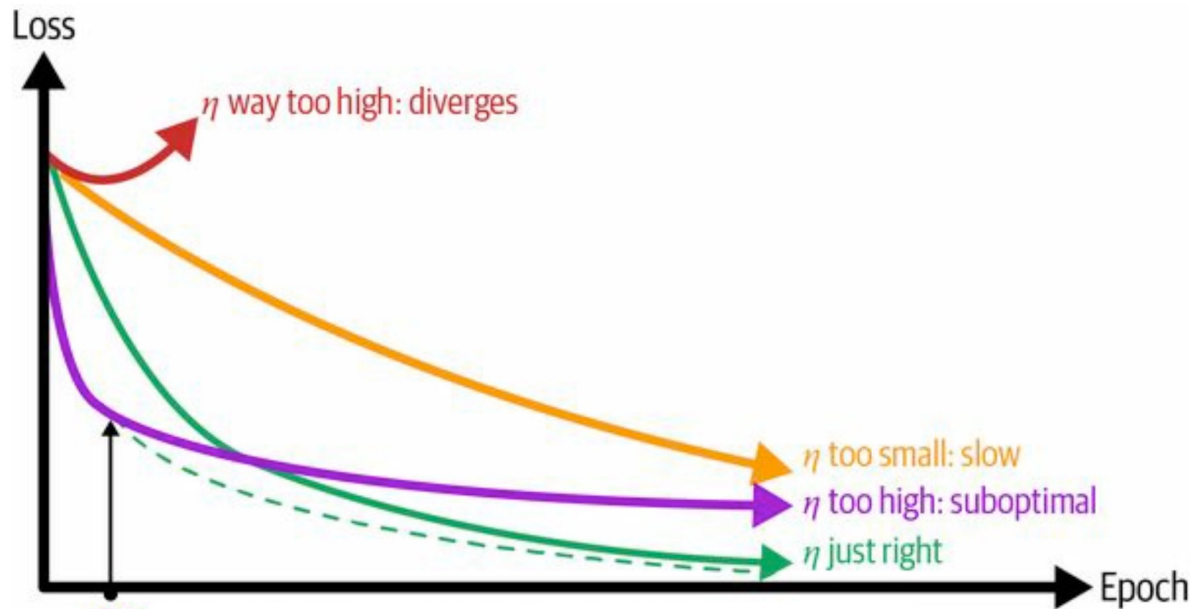
- One way to find a good learning rate is to train the model for a few hundred iterations, starting with a very low learning rate (e.g., 10^{-5}) and gradually increasing it up to a very large value (e.g., 10).
- This is done by multiplying the learning rate by a constant factor at each iteration (e.g., to go from 10^{-5} to 10 in 500 iterations).



Learning Rate Schedules

- A constant learning rate doesn't account for the possibility that the gradient landscape changes as we move across it.
 - What if instead, we adjust the learning rate according to a schedule?
-

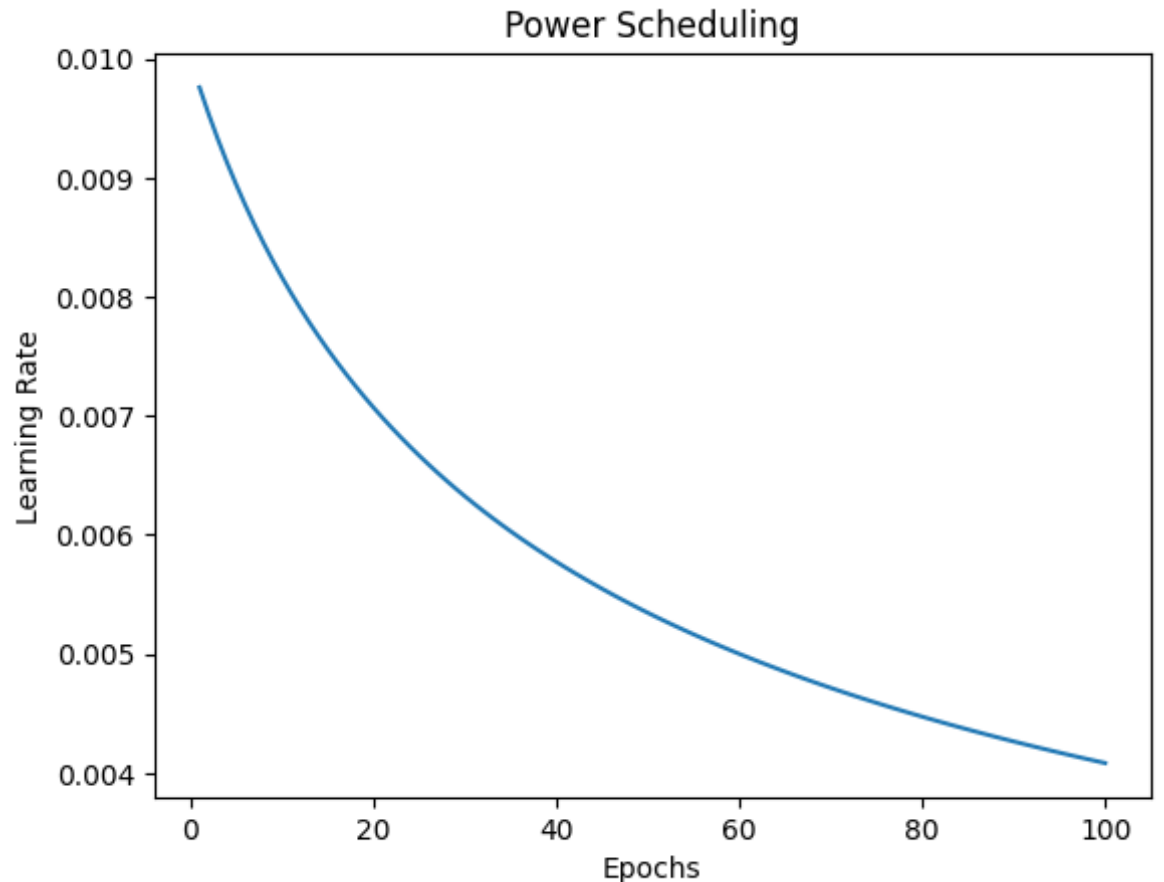
Motivation



1. **Early Training:** Initially, a larger learning rate can be beneficial, enabling rapid learning and helping the model to quickly escape suboptimal local minima.
2. **Later Stages:** As training progresses, reducing the learning rate helps to stabilize the learning process and fine-tune the model parameters, leading to better convergence.
3. **Preventing Overfitting and Oscillations:** A carefully chosen learning rate schedule can prevent the model from overfitting and reduce oscillations near the minima.

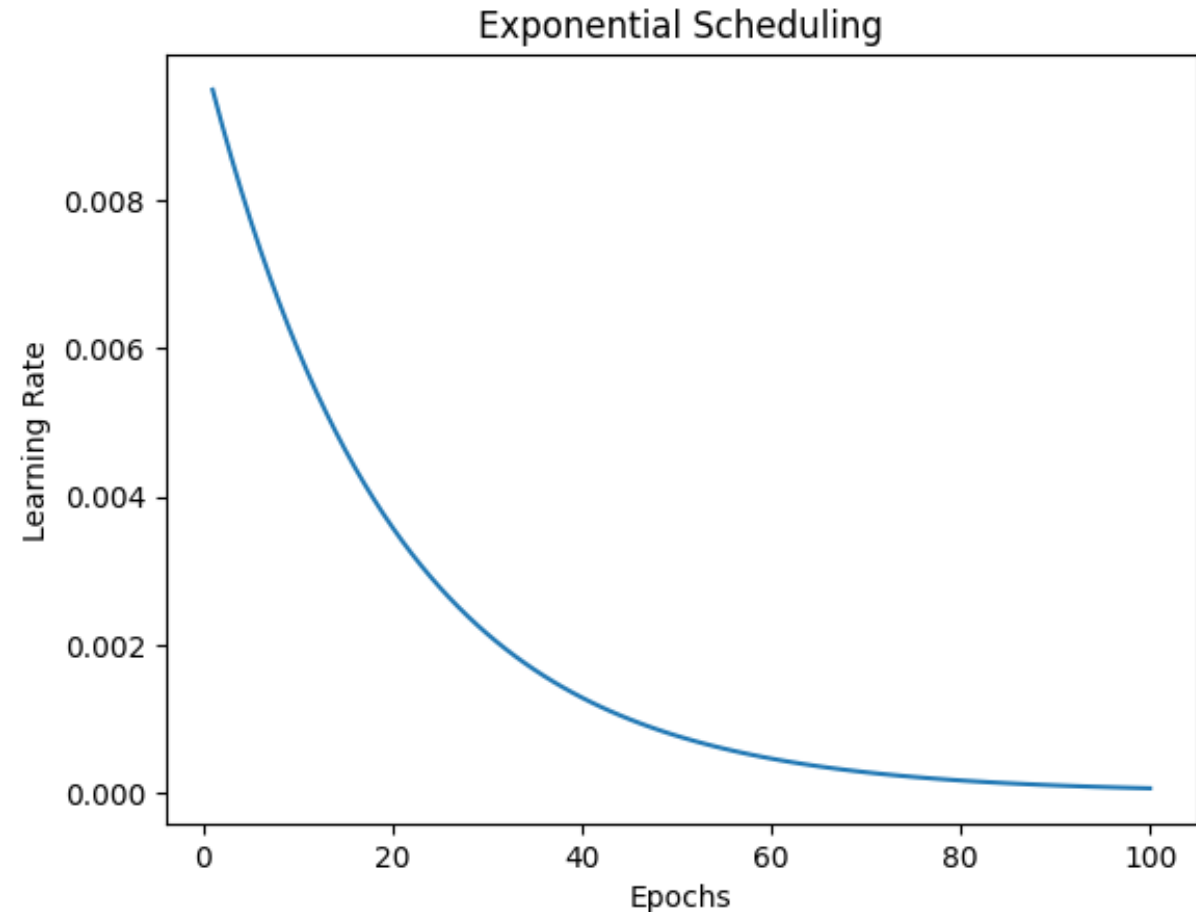
Methods: Power Scheduling

- Gradually decrease the learning rate over time using a polynomial decay, typically quadratic.
- $\eta(t) = \eta_0 / (1 + t/s)^c$



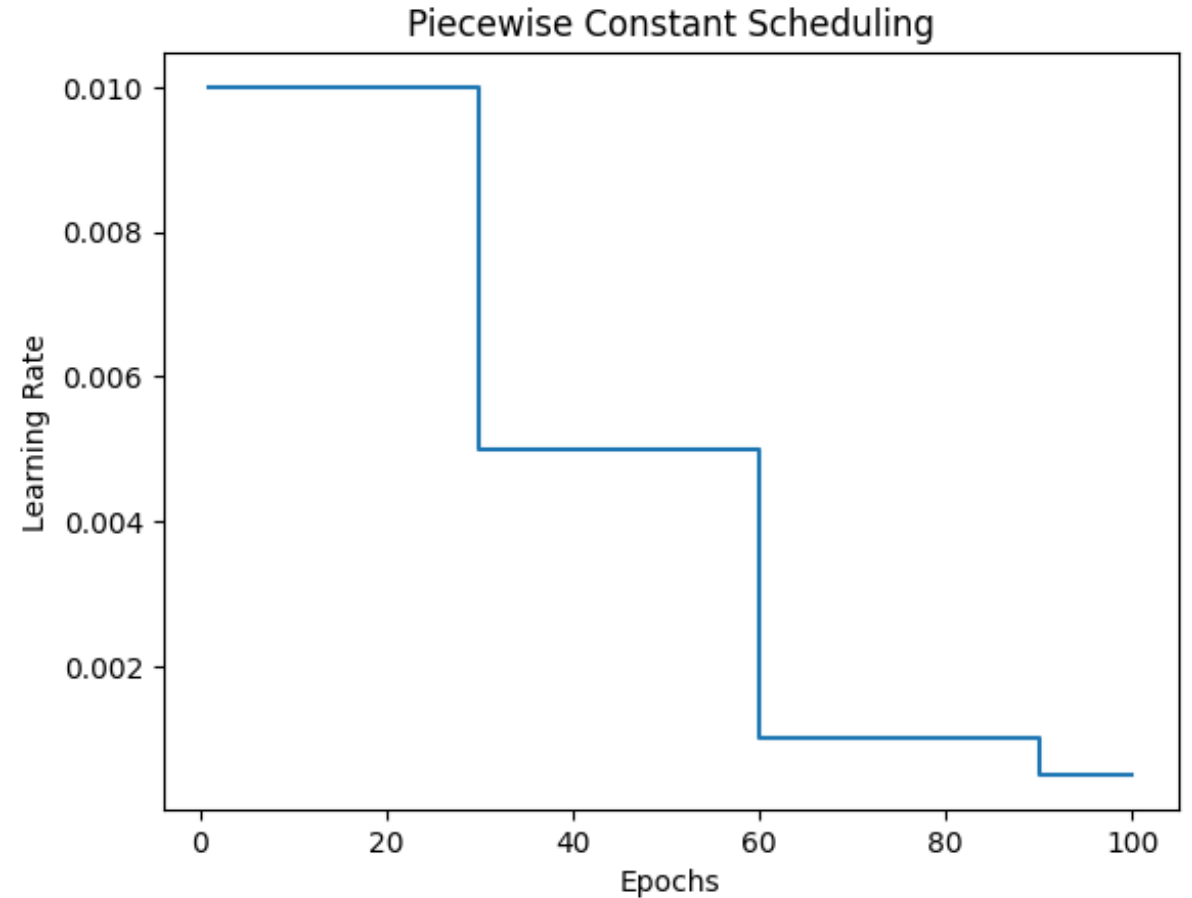
Methods: Exponential Scheduling

- Cut the learning rate by a factor of γ every s steps.
- $\eta(t) = \eta_0 \cdot \gamma^{t/s}$



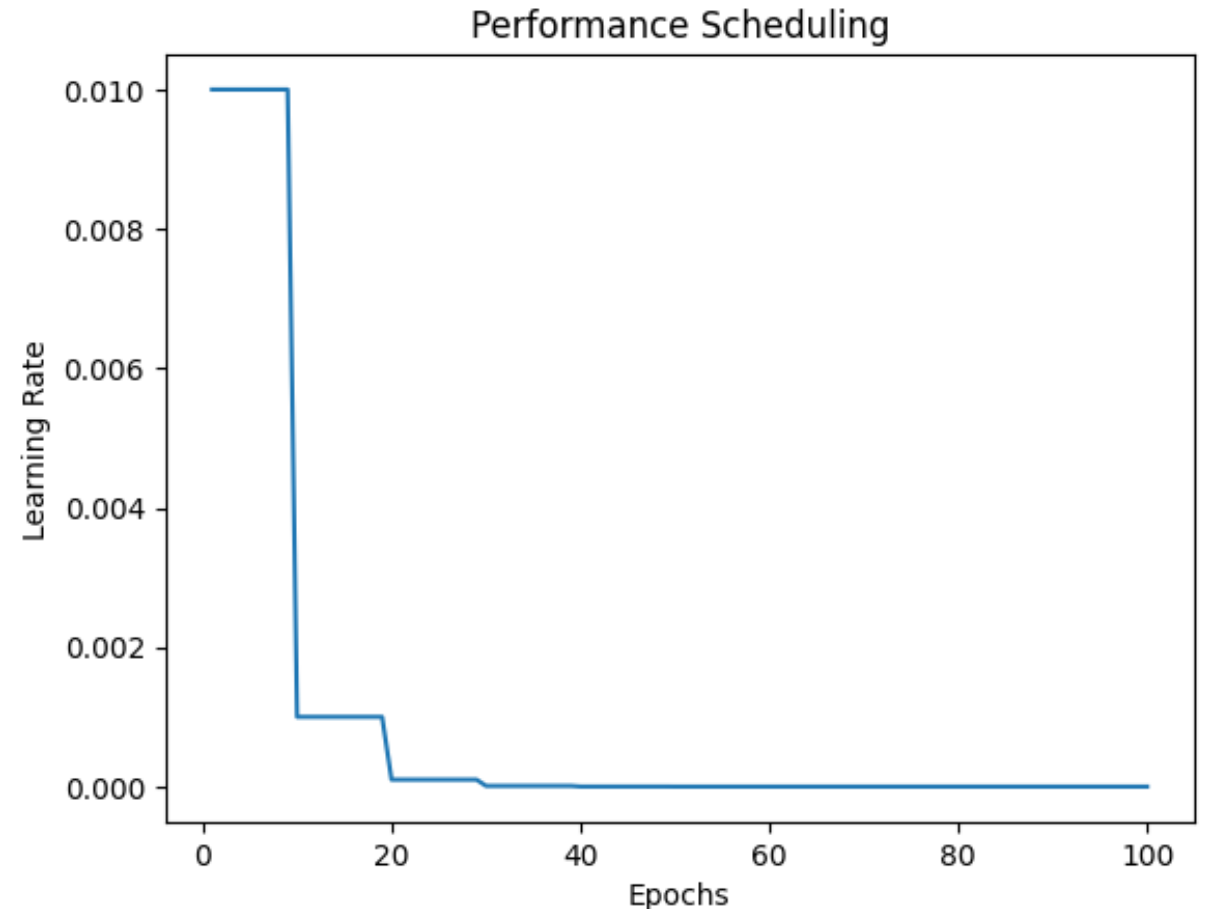
Methods: Piecewise Constant Scheduling

- Uses a constant learning rate for a set number of epochs and then lowers it to another constant rate for another set of epochs, and so on.



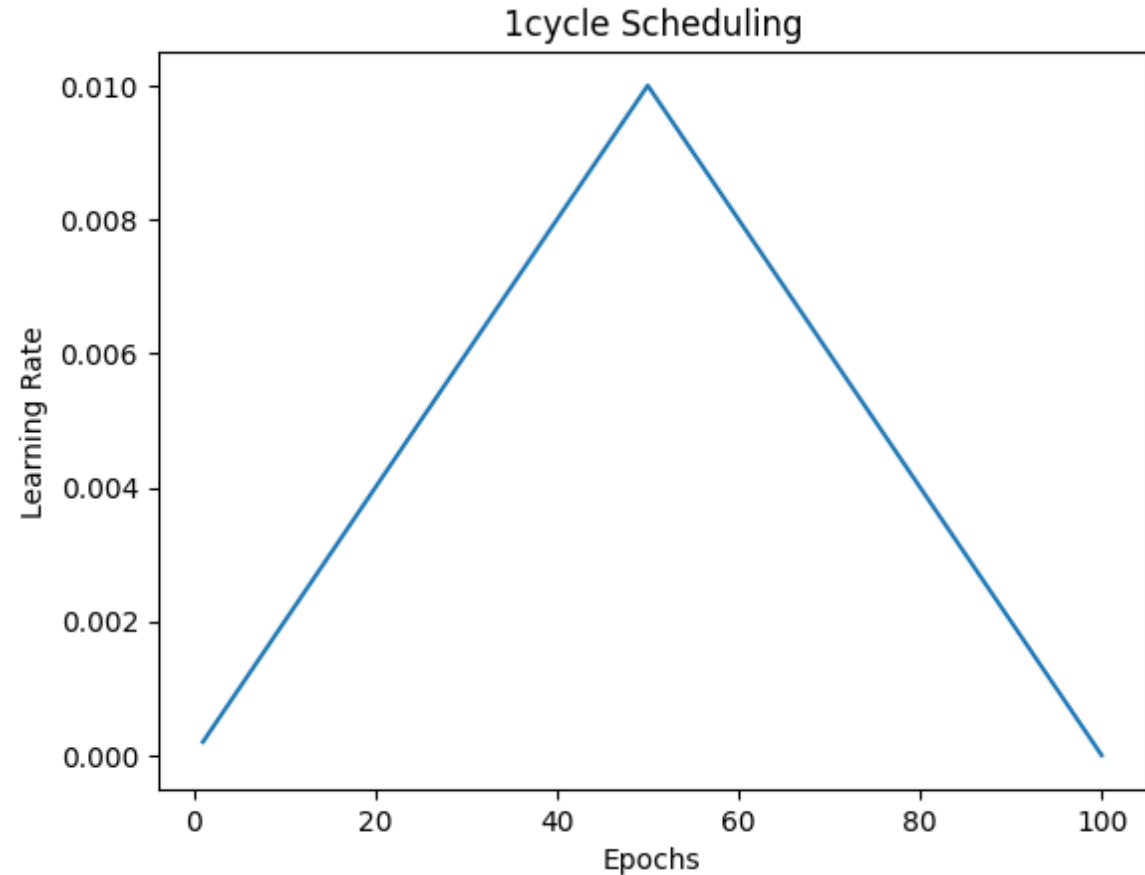
Methods: Performance Scheduling

- Measures the validation error every N steps (or epochs) and reduces the learning rate by λ if the error has stopped dropping.



Methods: 1cycle Scheduling

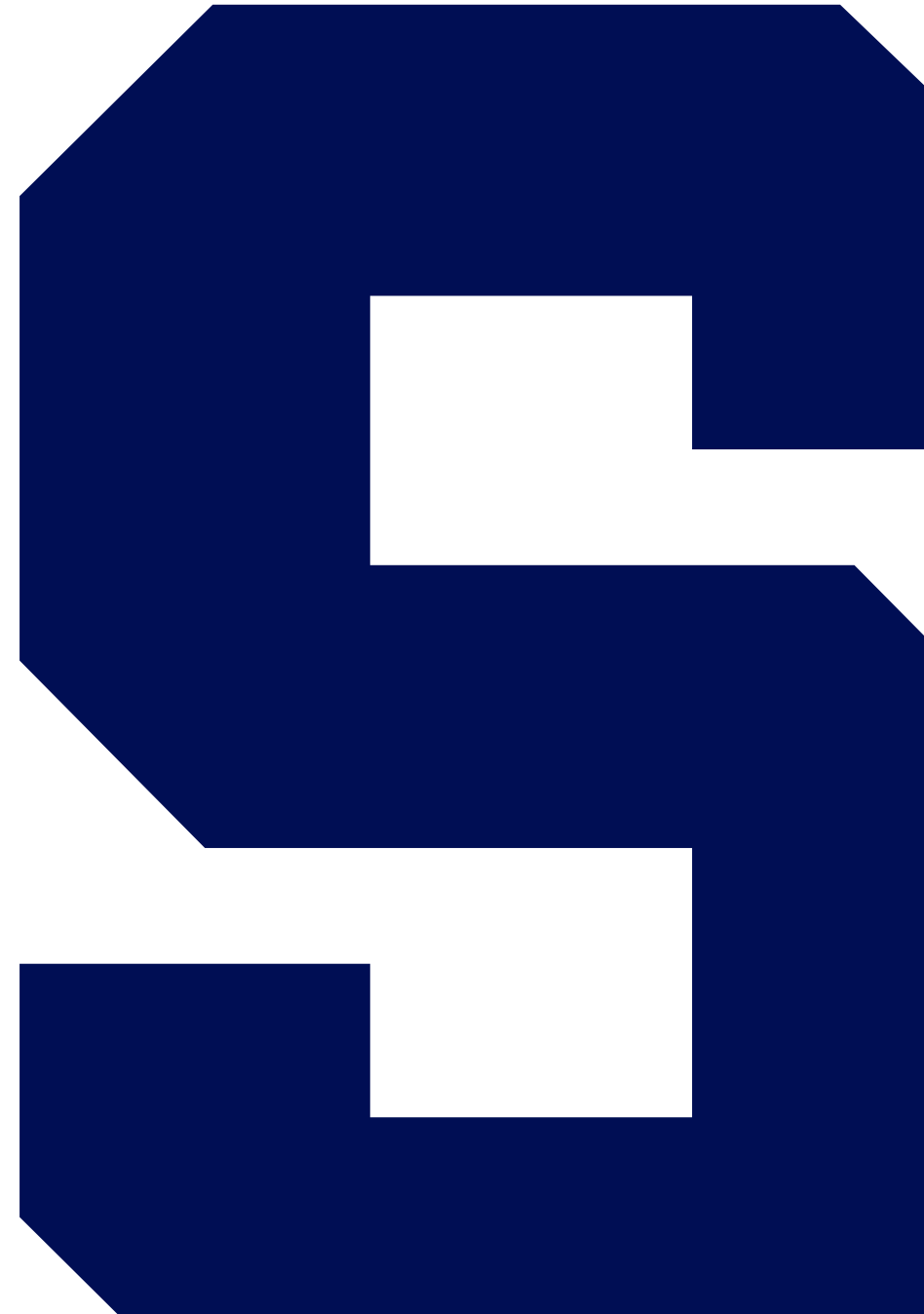
- Starts with a low learning rate, increases it linearly for the first half of training, and then decreases it linearly for the second half, possibly followed by a few epochs with a very small learning rate to finalize training.
- Shown to lead to faster convergence and better performance.



Wrap-up

- Learning rate scheduling is a powerful tool in the training of neural networks.
 - Different scheduling methods offer various advantages, and the choice of method can depend on the specific characteristics of the task, the dataset, and the desired training dynamics.
-

Optimizers

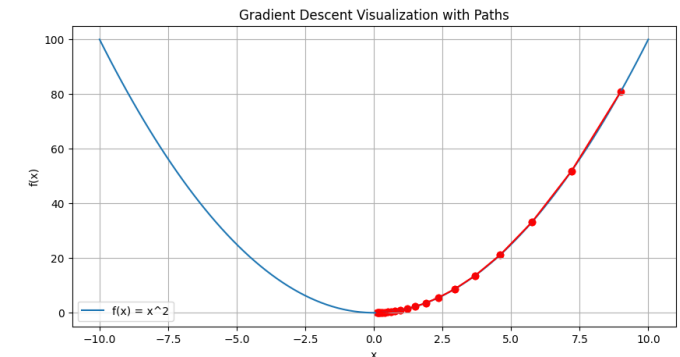


Why Optimization Matters

- The core objective of a neural network is to find the best possible values for its weights that minimize the loss function, leading to more accurate predictions. Optimization algorithms are the tools that make this search efficient and effective.
- Stochastic gradient descent is the foundation of neural network optimization. It updates the weights incrementally after each iteration over a batch, using the gradient of the loss function.
- SGD is simple and effective, but it can be slow and inefficient, especially in cases of irregular cost functions.

SGD

$$w_{new} = w - \alpha * \frac{\delta L}{\delta w}$$



Momentum Optimization

- Momentum optimization is a technique that builds upon Stochastic Gradient Descent (SGD) to accelerate the training process of a neural network, particularly in scenarios where the surface of the objective function has different curvatures in different dimensions.
- By accumulating momentum, the optimizer can traverse the parameter space faster than basic SGD, leading to quicker convergence.

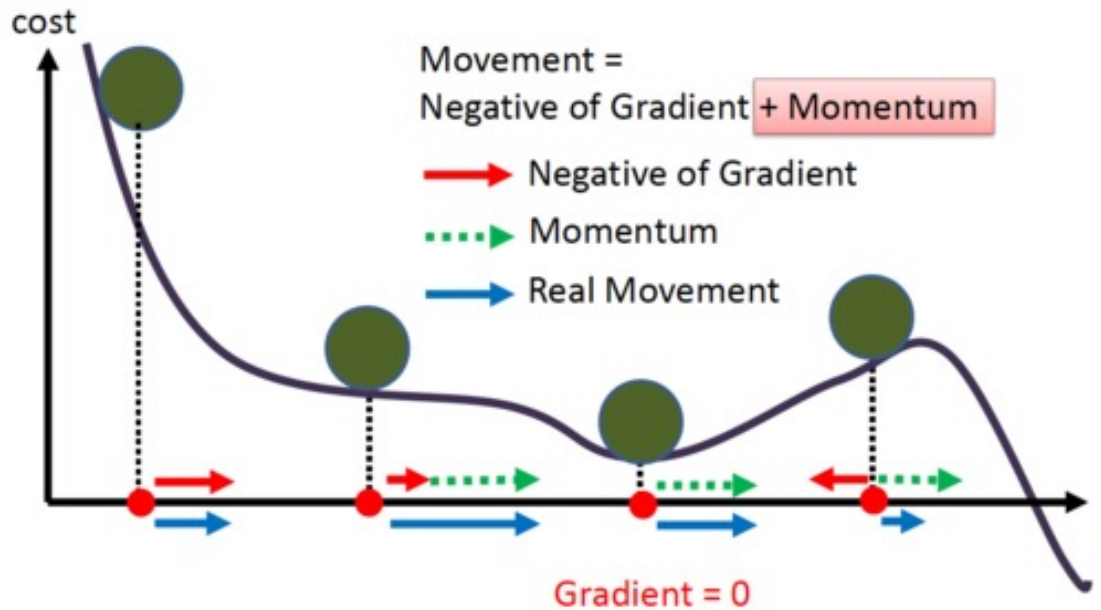


Image: <https://medium.com/analytics-vidhya/momentum-rmsprop-and-adam-optimizer-5769721b4b19>

Nesterov Accelerated Gradient

- NAG refines Momentum Optimization. In NAG, instead of calculating the gradient at the current position, the gradient is calculated a little ahead in the direction of the accumulated momentum.
- This approach allows the optimizer to have a more informed and potentially more accurate update, especially in valleys, ruts, or situations where the surface curves much.

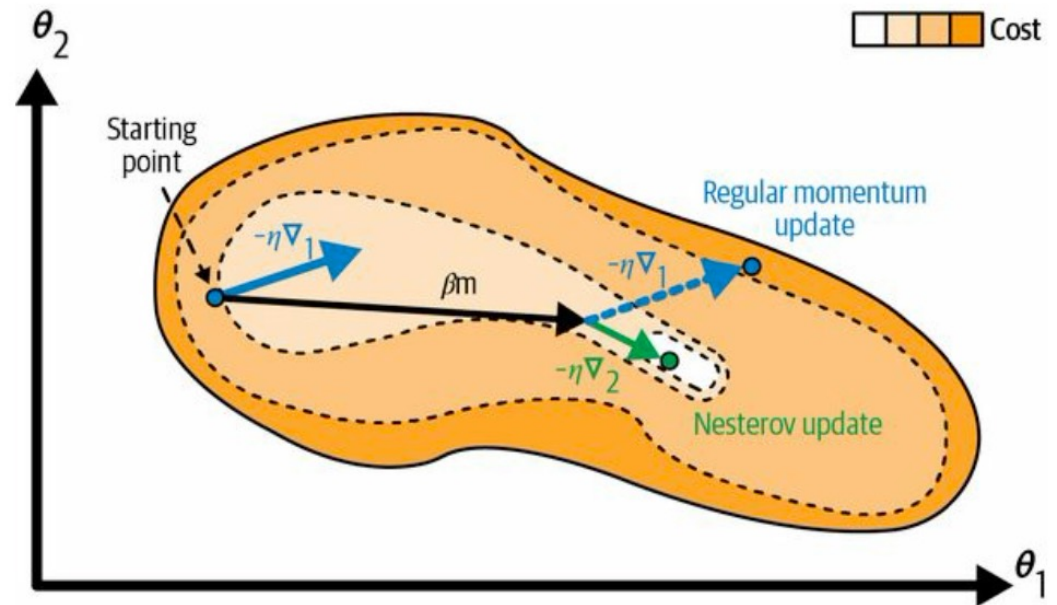
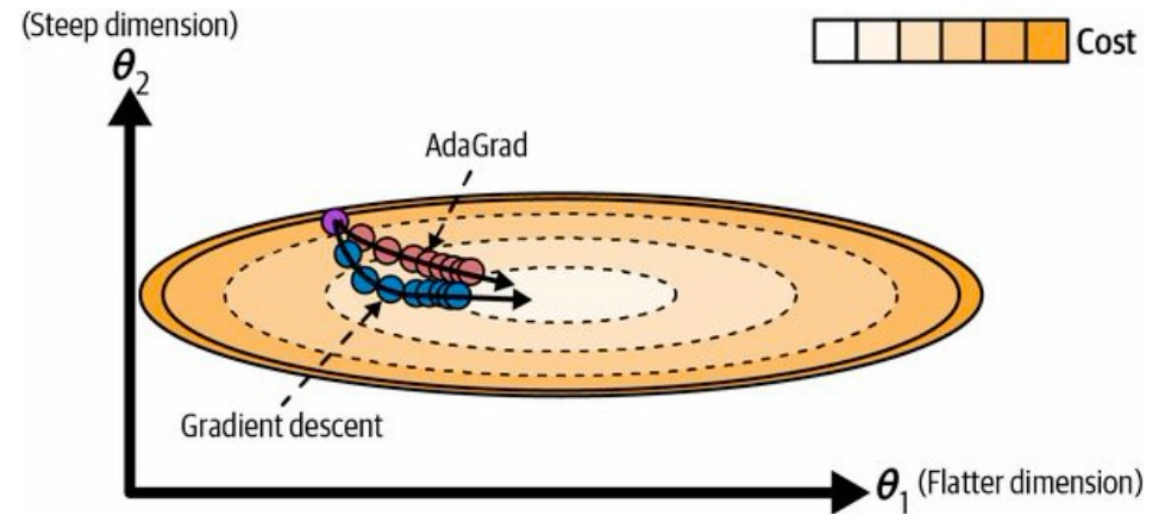


Image: Hands-On Machine Learning with Scikit-Learn, Keras & Tensorflow

AdaGrad

- AdaGrad (Adaptive Gradient) is an optimization algorithm that adapts the learning rates of all parameters.
- AdaGrad decays the learning rate, doing so faster for steep dimensions than for dimensions with gentler slopes.
- Limitation: Can lead to early stopping and should not generally be used in deep networks.

$$\mathbf{s} \leftarrow \mathbf{s} + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$$
$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \varepsilon}$$



RMSProp

- RMSProp (Root Mean Square Propagation) is designed to address the diminishing learning rates issue found in AdaGrad.
- Instead of accumulating *all* gradients during training, like Adagrad, RMSProp accumulates only the gradients from the most recent iterations, using exponential decay.

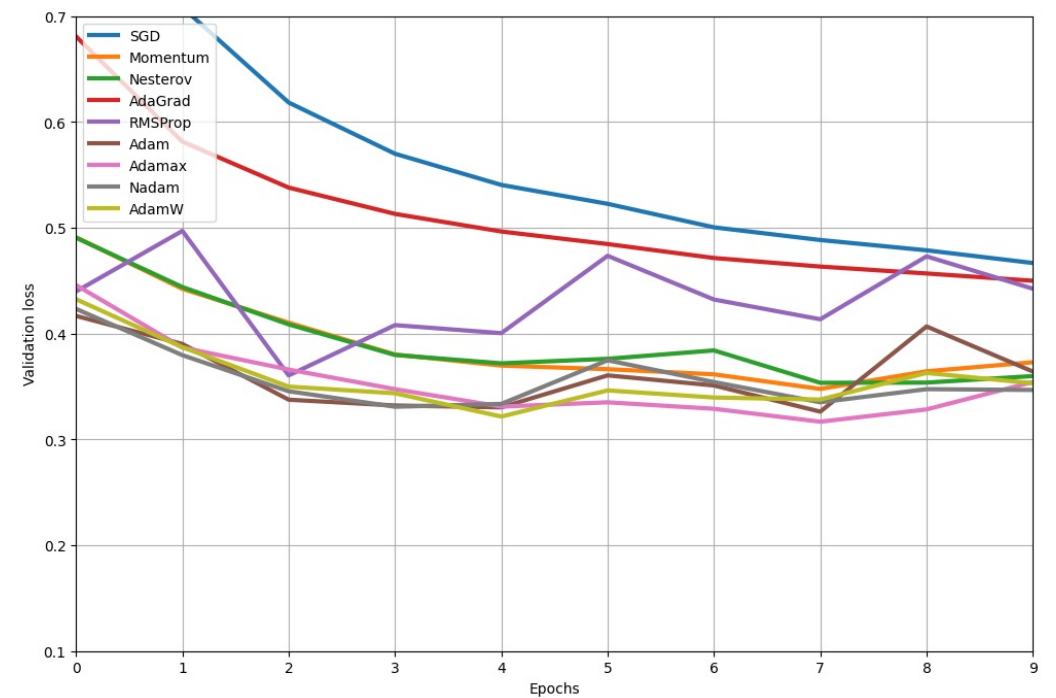
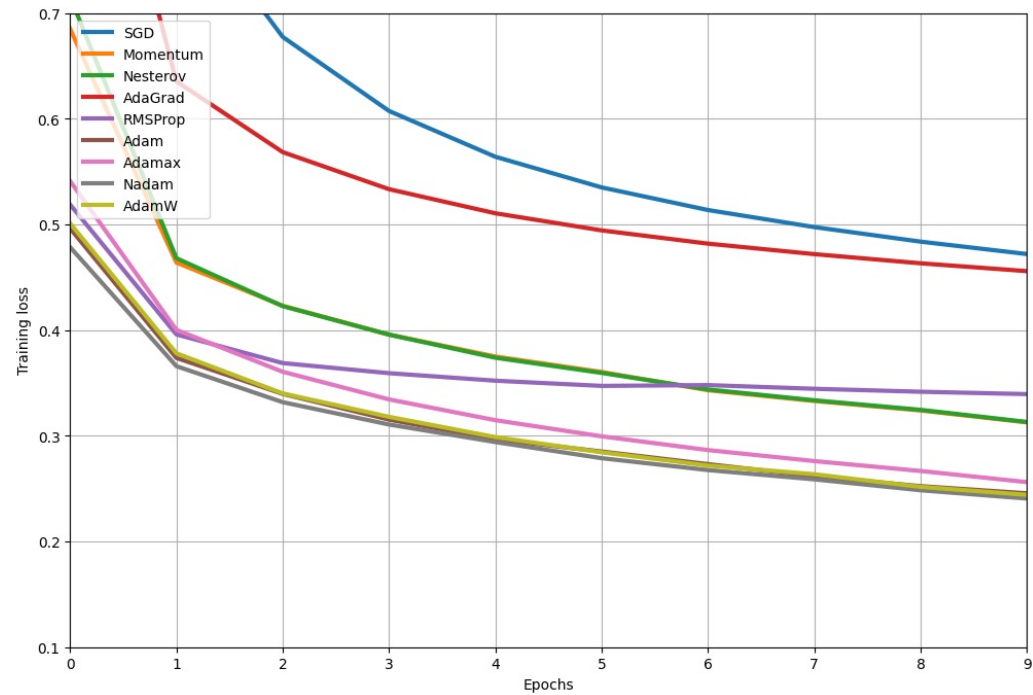
$$\begin{aligned}\mathbf{s} &\leftarrow \rho \mathbf{s} + (1 - \rho) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta) \\ \theta &\leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \varepsilon}\end{aligned}$$

Adam Optimization

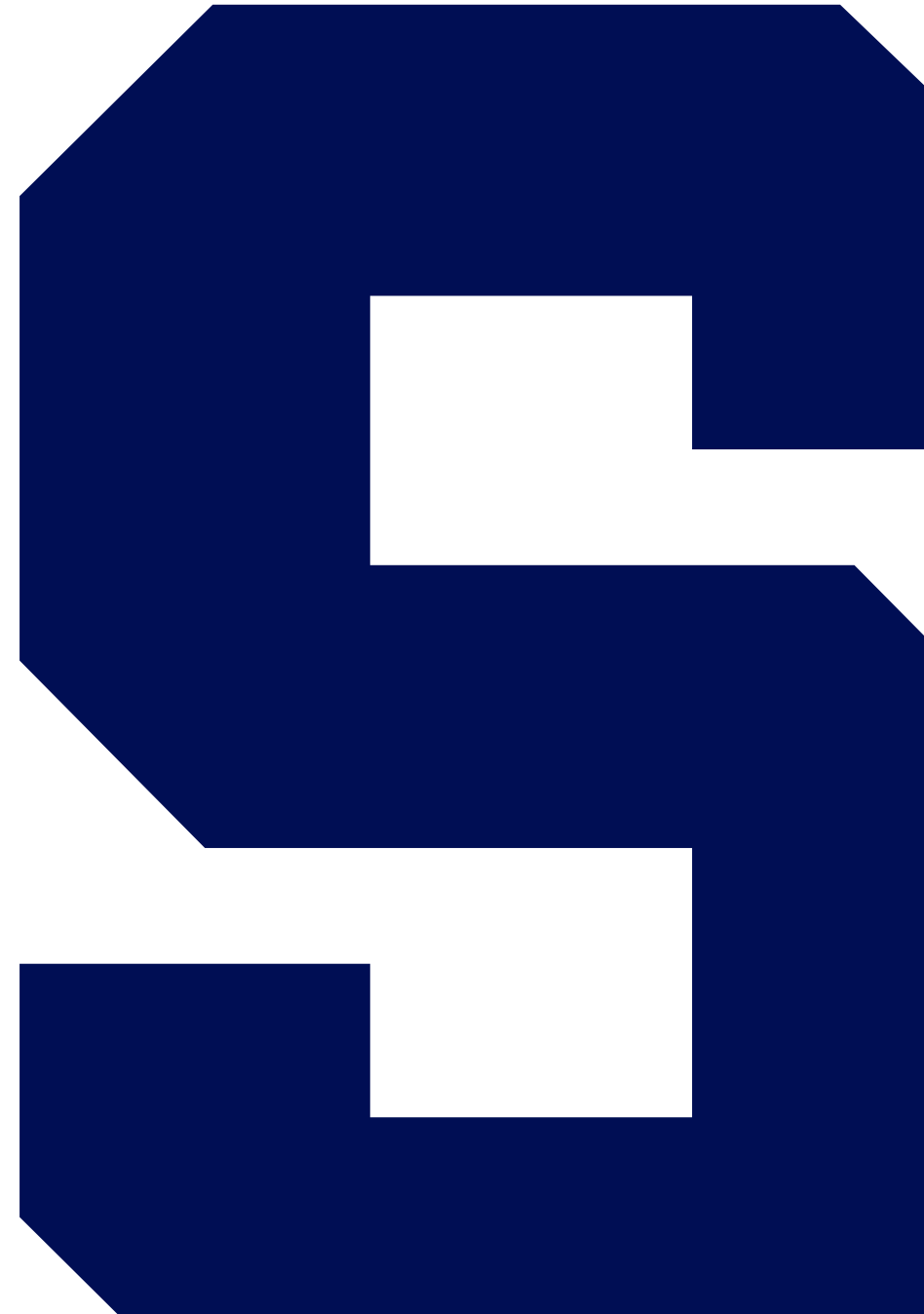
- Adam (**Adaptive Moment Estimation**) combines ideas from both RMSProp and Momentum Optimization, providing the benefits of adaptive learning rate optimization with momentum.
 - First Moment Vector: Running average of the gradients — like the "**mean**" of the gradients over time.
 - Second Moment Vector: Running average of the squares of the gradients — related to the "**variance**" of the gradients. This helps in scaling the gradient updates, making sure that the steps are not too large in directions with high variance (i.e., where the gradient changes rapidly)
 - Imagine navigating a landscape with a sophisticated GPS device that not only tells you how steep the hill is (like RMSProp) but also remembers your previous direction and speed (like Momentum).
 - Adam variants — Adamax, Nadam (Nesterov-accelerated Adaptive Moment Estimation), AdamW — modify the Adam algorithm and seek to improve it.
-

Optimization Summary

Different optimization algorithms used on the Fashion MNIST dataset for 10 epochs (3 hidden layers with 100 neurons each, ReLU activation).



Regularization



Motivation for Regularization

1. **Model Complexity:** Deep learning models, due to their high flexibility and capacity, can easily capture noise and random fluctuations in the training data. This leads to overfitting.
 2. **Generalization:** The goal of a model is not just to learn the training data but to make accurate predictions on new, unseen data. Regularization techniques help in achieving this by making the model less sensitive to individual fluctuations in the training data.
 3. **Improving Robustness:** Regularization can also improve the robustness of models, making them less likely to be influenced by small variations or corrupted inputs.
-

L1 Regularization (Lasso)

Concept:

- L1 regularization adds a penalty equal to the absolute value of the magnitude of coefficients to the loss function.
- The L1 regularized loss for a linear model: **Loss = MSE + $\lambda \sum |w_i|$**

Sparsity:

- Tends to produce sparse models, where some coefficients can become exactly zero.
- This is useful for feature selection, making L1 regularization a tool for identifying important features in the data.

Use Cases:

- Particularly beneficial in scenarios where some features are irrelevant or redundant.
-

L2 Regularization (Ridge)

Concept:

- L2 regularization adds a penalty equal to the square of the magnitude of coefficients.
- The L2 regularized loss for a linear model: **Loss = MSE + $\lambda \sum w_i^2$** .

Weight Shrinkage:

- Unlike L1, L2 regularization doesn't lead to sparse models.
- Instead, it shrinks the weights towards zero but rarely reaches zero.
- This leads to smaller and more distributed weight values, which helps in preventing overfitting.

Use Cases:

- Ideal for problems where all features have relevance or when you have correlated features.
-

Extending L1/L2 Regularization to Deep Learning

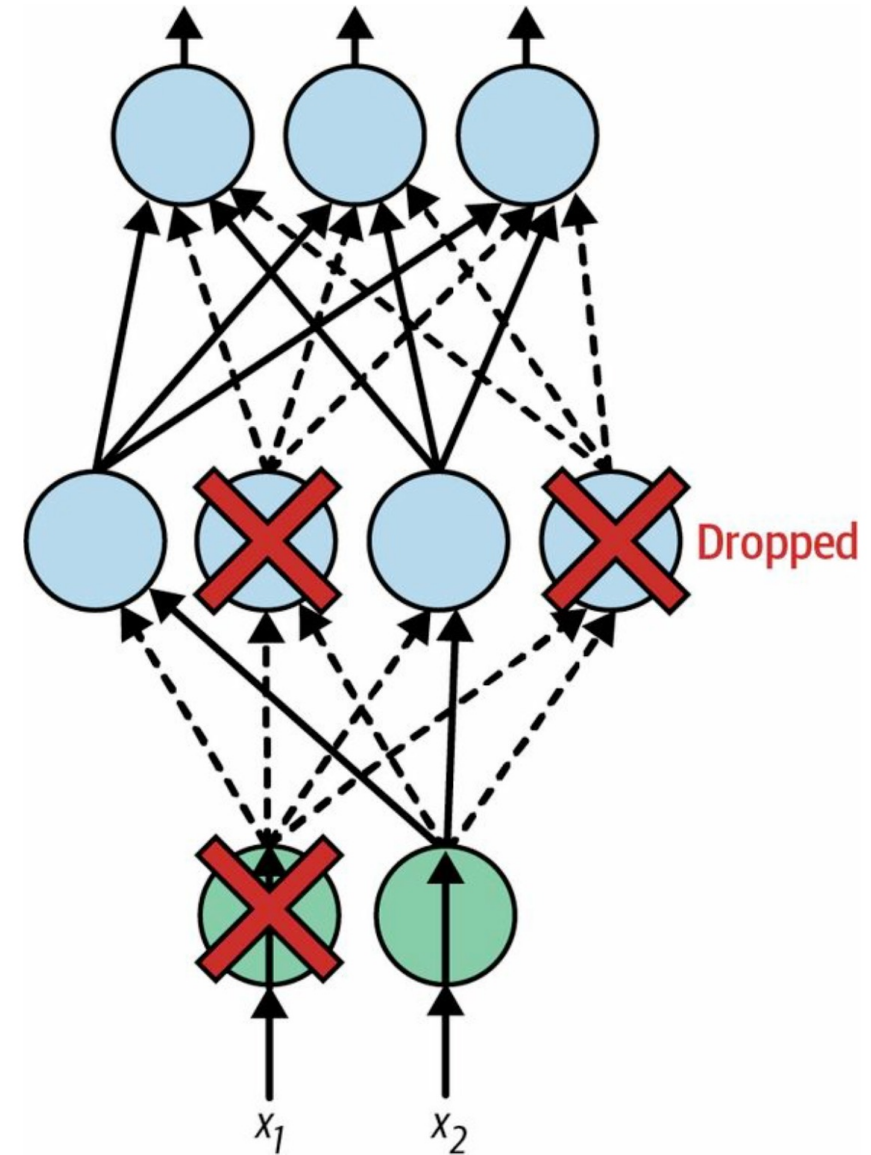
- L1 and L2 regularization in neural networks involve adding a **penalty term** to the loss function, based on the weights of the network layers.
 - **Choice Between L1 and L2:** depends on the network architecture and the nature of the data. L2 (ridge) is more common in deep learning due to its tendency to encourage smaller weights, thus smoothing the learned function. However, L1 (lasso) can lead to sparser networks, which require less memory and can run faster.
 - **Hyperparameter Tuning:** The regularization parameter λ controls the trade-off between fitting the training data and keeping the model weights small, and it's usually tuned using cross-validation.
 - **Impact on Learning:** It's important to monitor both training and validation performance to understand the impact and adjust the regularization strength accordingly. Note: L2 works fine with SGD, momentum optimization, and Nesterov momentum, but not Adam or its variants.
-

Dropout

- Dropout is a remarkably effective and widely used regularization technique in deep neural networks, introduced by Geoffrey Hinton et al (2012) and further detailed by Nitish Srivastava et al (2014).
 - Dropout's simplicity and effectiveness in enhancing network generalization have made it a staple in many state-of-the-art architectures.
-

The Basic Idea of Dropout

- During training, dropout randomly "drops out" (i.e., temporarily deactivates) neurons in the network with a certain probability p , known as the dropout rate.
- This process is applied at every training step, and different neurons can be dropped in each step.



Why Does Dropout Work?

- **Redundancy Reduction:** Just like a company diversifying skillsets among employees, dropout forces the network to spread "knowledge" across its neurons, making it more robust and less reliant on specific neurons.
 - **Prevents Co-Adaptation:** Neurons become less sensitive to specific features or patterns in the input data, leading to a model that generalizes better.
 - **Ensemble Interpretation:** Conceptually, each training step with dropout is training a different "thinned" network. The final model can be seen as an averaging ensemble of these various networks.
-

Dropout Application in Deep Learning

- **Use in Different Layers:** Dropout can be applied after dense layers, convolutional layers, or even recurrent layers, though the dropout rate might need adjustment depending on the layer type and the overall network architecture. In practice, you can usually apply dropout only to the neurons in the top one to three layers (excluding the output layer).
 - **Balancing Dropout Rate:** The choice of dropout rate is crucial; too high, and the network might underfit; too low, and the regularization effect might be insufficient. This rate is often determined through cross-validation.
 - **Evaluation:** Since dropout is only active during training, comparing the training loss and the validation loss can be misleading. In particular, a model may be overfitting the training set and yet have similar training and validation losses. Make sure to evaluate the training loss without dropout (e.g., after training).
-

Wrap-Up

- Regularization in deep learning is a crucial technique to prevent overfitting, where a model performs well on training data but poorly on unseen data.
 - Early stopping and batch normalization.
 - L1 & L2 regularization, Dropout, and Max-Norm.
 - Reading and demos provide good practical guidelines.
 - Hyperparameter tuning: learning rate, number of layers, activation functions, batch size, optimizer, dropout rate.
-